DBMS – DATABASE RELATIONSHIP ANALYSIS USING JOINS

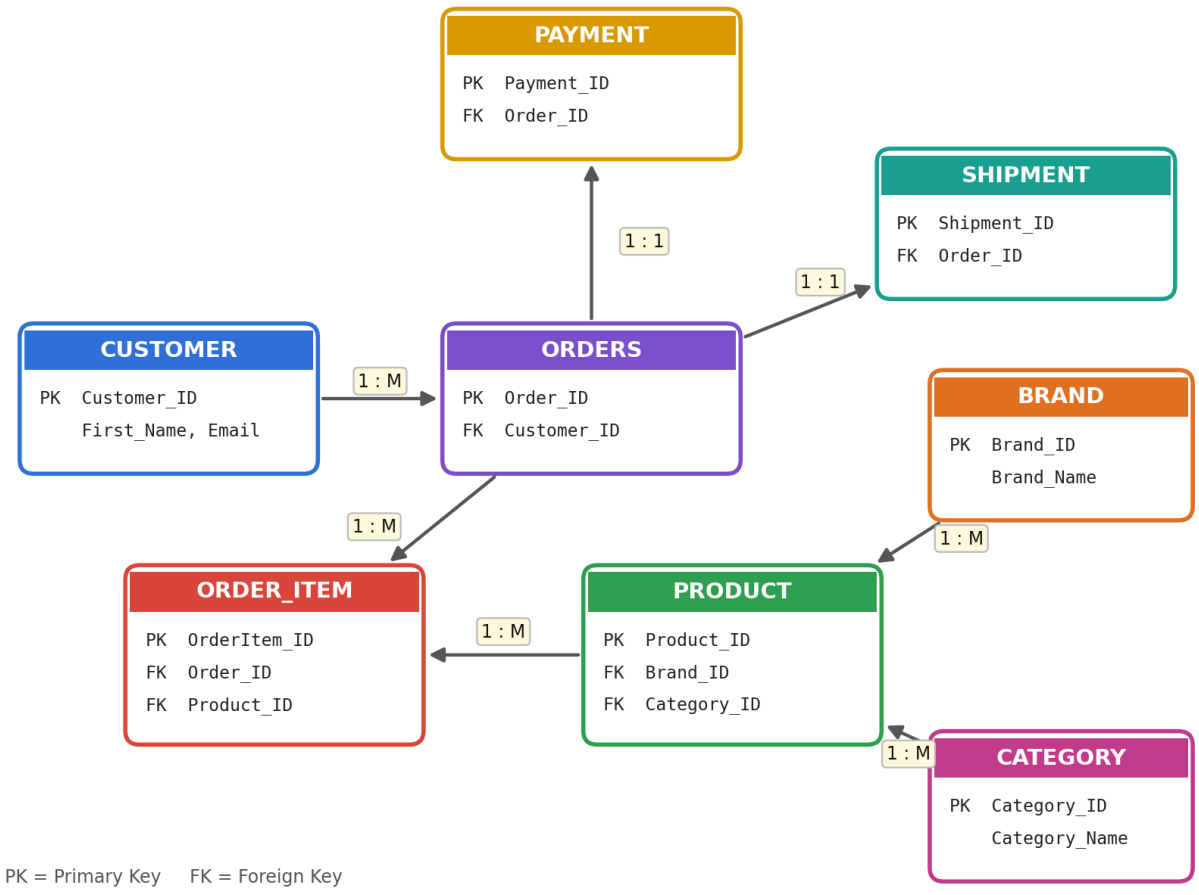
Objectives

1. Combine Customer, Product, Orders, and Payment tables.
2. Implement INNER JOIN, LEFT JOIN, and RIGHT JOIN.
3. Retrieve complete order details.
4. Display customer purchase history.
5. Generate multi-table reports.

1. Combining Multiple Tables

The Myntra database contains different tables that store customer, product, brand, category, order, payment, and shipment information.

These tables are connected through primary key and foreign key columns, and they can be combined using **JOIN operations** to retrieve related information.



2. INNER JOIN

INNER JOIN returns only the records that have matching values in both tables.

2.1 Customer and Orders

This displays customers who have placed orders.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      o.Total_Amount
  7  FROM Customer c
  8  INNER JOIN Orders o
  9      ON c.Customer_ID = o.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME   ORDER_ID ORDER_DAT TOTAL_AMOUNT
----------- --------------- -------- --------- ------------
        101 Anitha Kumar         201 16-SEP-26         3500
        102 Rahul Sharma         202 15-SEP-26         1800
        103 Priya Nair           203 12-SEP-26         3200
```

2.2 Orders and Payment

This displays orders and their corresponding payment details.

```
SQL> SELECT
  2      o.Order_ID,
  3      o.Order_Date,
  4      o.Total_Amount,
  5      p.Payment_Method,
  6      p.Amount AS Payment_Amount,
  7      p.Payment_Status
  8  FROM Orders o
  9  INNER JOIN Payment p
 10      ON o.Order_ID = p.Order_ID;

ORDER_ID ORDER_DAT TOTAL_AMOUNT PAYMENT_METHOD PAYMENT_AMOUNT PAYMENT_STATUS
-------- --------- ------------ -------------- -------------- --------------
     201 16-SEP-26         3500 UPI                      3500 Successful
     202 15-SEP-26         1800 Card                     1800 Successful
     203 12-SEP-26         3200 Net Banking              3200 Successful
```

2.3 Product and Order_Item

This displays products that have been included in orders.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      oi.Order_ID,
  5      oi.Quantity,
  6      oi.Unit_Price,
  7      oi.Quantity * oi.Unit_Price AS Subtotal
  8  FROM Product p
  9  INNER JOIN Order_Item oi
 10      ON p.Product_ID = oi.Product_ID;

PRODUCT_ID PRODUCT_NAME           ORDER_ID QUANTITY UNIT_PRICE SUBTOTAL
---------- ---------------------- -------- -------- ---------- --------
       501 Slim Fit Cotton Shirt       201        2       1000     2000
       502 Denim Jeans                 201        1       1500     1500
       503 Running Shoes               202        1       1800     1800
       504 Kurta Set                   203        2       1600     3200
```

3. LEFT JOIN

LEFT JOIN returns all records from the left table and the matching records from the right table. Where there is no match, the right-side columns are displayed as NULL (blank).

3.1 Display All Customers and Their Orders

This displays **all customers**, including customers who have not placed an order.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      o.Total_Amount
  7  FROM Customer c
  8  LEFT JOIN Orders o
  9      ON c.Customer_ID = o.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME   ORDER_ID ORDER_DAT TOTAL_AMOUNT
----------- --------------- -------- --------- ------------
        101 Anitha Kumar         201 16-SEP-26         3500
        102 Rahul Sharma         202 15-SEP-26         1800
        103 Priya Nair           203 12-SEP-26         3200
        104 Karthik Raj
```

3.2 Display Products That Have Never Been Ordered

This displays products that do not appear in any order.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      p.Price
  5  FROM Product p
  6  LEFT JOIN Order_Item oi
  7      ON p.Product_ID = oi.Product_ID
  8  WHERE oi.OrderItem_ID IS NULL;

PRODUCT_ID PRODUCT_NAME              PRICE
---------- ---------------------- --------
       505 Leather Wallet              799
```

4. RIGHT JOIN

RIGHT JOIN returns all records from the right table and the matching records from the left table.

4.1 Display All Orders and Customer Details

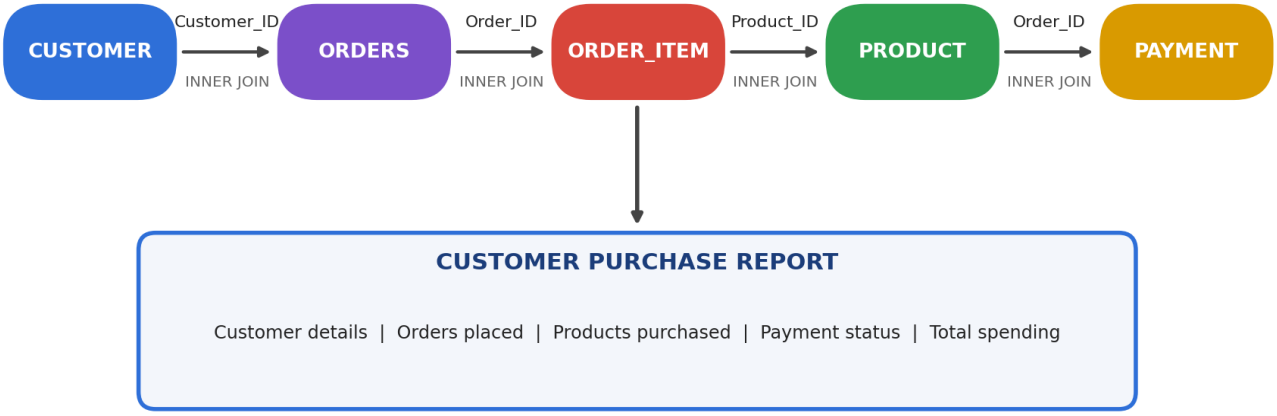
This displays **all orders** along with the matching customer information.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      o.Total_Amount
  7  FROM Customer c
  8  RIGHT JOIN Orders o
  9      ON c.Customer_ID = o.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME   ORDER_ID ORDER_DAT TOTAL_AMOUNT
----------- --------------- -------- --------- ------------
        101 Anitha Kumar         201 16-SEP-26         3500
        102 Rahul Sharma         202 15-SEP-26         1800
        103 Priya Nair           203 12-SEP-26         3200
```

5. Complete Order Details

This query combines the **Customer, Orders, Order_Item, Product, Brand, Category, and Payment** tables.




```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      p.Product_ID,
  7      p.Product_Name,
  8      b.Brand_Name,
  9      ca.Category_Name,
 10      oi.Quantity,
 11      oi.Unit_Price,
 12      oi.Quantity * oi.Unit_Price AS Subtotal,
 13      py.Payment_Method,
 14      py.Amount AS Payment_Amount,
 15      py.Payment_Status
 16  FROM Customer c
 17  INNER JOIN Orders o
 18      ON c.Customer_ID = o.Customer_ID
 19  INNER JOIN Order_Item oi
 20      ON o.Order_ID = oi.Order_ID
 21  INNER JOIN Product p
 22      ON oi.Product_ID = p.Product_ID
 23  INNER JOIN Brand b
 24      ON p.Brand_ID = b.Brand_ID
 25  INNER JOIN Category ca
 26      ON p.Category_ID = ca.Category_ID
 27  INNER JOIN Payment py
 28      ON o.Order_ID = py.Order_ID
 29  ORDER BY o.Order_Date DESC;
```

6. Customer Purchase History

This query displays the products purchased by each customer.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      p.Product_Name,
  7      oi.Quantity,
  8      oi.Quantity * oi.Unit_Price AS Subtotal
  9  FROM Customer c
 10  INNER JOIN Orders o
 11      ON c.Customer_ID = o.Customer_ID
 12  INNER JOIN Order_Item oi
 13      ON o.Order_ID = oi.Order_ID
 14  INNER JOIN Product p
 15      ON oi.Product_ID = p.Product_ID
 16  ORDER BY c.Customer_ID, o.Order_Date;

CUSTOMER_ID CUSTOMER_NAME  ORDER_ID ORDER_DAT PRODUCT_NAME          QUANTITY SUBTOTAL
----------- -------------- -------- --------- ---------------------- -------- --------
        101 Anitha Kumar        201 16-SEP-26 Slim Fit Cotton Shirt        2     2000
        101 Anitha Kumar        201 16-SEP-26 Denim Jeans                  1     1500
        102 Rahul Sharma        202 15-SEP-26 Running Shoes                1     1800
        103 Priya Nair          203 12-SEP-26 Kurta Set                    2     3200
```

7. Customer Purchase History with Payment

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4      o.Order_ID,
  5      o.Order_Date,
  6      p.Product_Name,
  7      oi.Quantity,
  8      oi.Quantity * oi.Unit_Price AS Subtotal,
  9      py.Payment_Method,
 10      py.Payment_Status
 11  FROM Customer c
 12  INNER JOIN Orders o
 13      ON c.Customer_ID = o.Customer_ID
 14  INNER JOIN Order_Item oi
 15      ON o.Order_ID = oi.Order_ID
 16  INNER JOIN Product p
 17      ON oi.Product_ID = p.Product_ID
 18  INNER JOIN Payment py
 19      ON o.Order_ID = py.Order_ID
 20  ORDER BY c.Customer_ID;

CUSTOMER_ID CUSTOMER_NAME ORDER_ID ORDER_DAT PRODUCT_NAME          QUANTITY SUBTOTAL PAYMENT_METHOD PAYMENT_STATUS
----------- ------------- -------- --------- ---------------------- -------- -------- -------------- --------------
        101 Anitha Kumar       201 16-SEP-26 Slim Fit Cotton Shirt        2     2000 UPI            Successful
        101 Anitha Kumar       201 16-SEP-26 Denim Jeans                  1     1500 UPI            Successful
        102 Rahul Sharma       202 15-SEP-26 Running Shoes                1     1800 Card           Successful
        103 Priya Nair         203 12-SEP-26 Kurta Set                    2     3200 Net Banking    Successful
```

8. Multi-Table Business Report

The following query generates a complete customer purchase report.

```
SQL> SELECT
  2      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3      COUNT(DISTINCT o.Order_ID) AS Total_Orders,
  4      SUM(oi.Quantity * oi.Unit_Price) AS Total_Purchase,
  5      SUM(oi.Quantity) AS Total_Products
  6  FROM Customer c
  7  INNER JOIN Orders o
  8      ON c.Customer_ID = o.Customer_ID
  9  INNER JOIN Order_Item oi
 10      ON o.Order_ID = oi.Order_ID
 11  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
 12  ORDER BY Total_Purchase DESC;

CUSTOMER_NAME   TOTAL_ORDERS TOTAL_PURCHASE TOTAL_PRODUCTS
--------------- ------------ -------------- --------------
Anitha Kumar               1           3500              3
Priya Nair                 1           3200              2
Rahul Sharma               1           1800              1
```

9. Payment Transaction Report

```
SQL> SELECT
  2      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3      o.Order_ID,
  4      o.Order_Date,
  5      o.Total_Amount,
  6      py.Payment_Method,
  7      py.Amount AS Payment_Amount,
  8      py.Payment_Status
  9  FROM Customer c
 10  INNER JOIN Orders o
 11      ON c.Customer_ID = o.Customer_ID
 12  INNER JOIN Payment py
 13      ON o.Order_ID = py.Order_ID
 14  ORDER BY o.Order_Date DESC;

CUSTOMER_NAME   ORDER_ID ORDER_DAT TOTAL_AMOUNT PAYMENT_METHOD PAYMENT_AMOUNT PAYMENT_STATUS
--------------- -------- --------- ------------ -------------- -------------- --------------
Anitha Kumar         201 16-SEP-26         3500 UPI                      3500 Successful
Rahul Sharma         202 15-SEP-26         1800 Card                     1800 Successful
Priya Nair           203 12-SEP-26         3200 Net Banking              3200 Successful
```